

Securing Apache

Intended for information security experts, systems administrators, and all those concerned about Web security, this third article in the series moves on from SQL injection and XSS to how to secure Web applications against Cross-Site Request Forgery attacks. I once again stress that neither LFY nor I aim to teach readers how to attack servers. Rather, the attack techniques are meant to give you the knowledge that you need in order to protect your own infrastructure.

Cross-Site Request Forgery (CSRF, a.k.a. XSRF, one-click attacks, session riding, confused deputy, client-side Trojan, hostile linking, automation attack or sea surf) is a client-side Web application attack, where an attacker exploits implicit authentication mechanisms to force an end user to execute unwanted actions in an authenticated Web application. CSRF is a non-avoidable dangerous security risk, and its constant 5th position in OWASP's Top 10 most critical security risks for the past three years, underlines this fact.

A little social engineering (for example, through e-mail or IM) is all it needs to trap a victim into unwittingly executing actions of the attacker's choice. A successful CSRF exploit can compromise end-user data and operations, when it targets a normal user. If targeted at an administrator account, the attack can compromise the entire Web application. The word CSRF itself means that the attack is done using a cross-site request; it's 'forged' because it's invisible to the user. A cross-site request is one where a page loaded from Site A makes a request to another site for resources that are part of the page (like images, for example). While it's easy for a malicious site to have such HTML code in its pages, such cross-site requests can also be caused by viewing bulletin boards, forums, or social networking sites (for example) where users are allowed to post images with foreign URL sources—that is, the images are hosted on other sites. CSRF attacks are effective in situations which meet the following criteria:

- The victim has an active session on the target site.
- The victim is authenticated by implicit authentication

mechanisms (like cookies or HTTP authentication) on the target site.

The main aim of a CSRF attack is to perform an action against the target site using the victim's privileges in a properly authenticated session. The key requirement is prior access to, and knowledge of the Web application's valid URLs, by the attacker. In a typical CSRF attack, the attacker creates a hidden HTTP request inside the victim's Web browser, which is executed in the victim's authentication context, without his knowledge.

Three sample CSRF scenarios

We will now look at three sample scenarios in which CSRF is possible, and how the attack could be carried out.

Scenario 1

Assume the victim is logged in to his online bank account, and is using the money transfer feature. The bank's site uses cookies (implicit authentication) to authenticate logged-in user sessions. After filling out the money transfer form, when the *Submit/OK/Transfer* button is clicked, the victim's Web browser issues an HTTP request to the bank's Web server, to perform the transfer. Here's what the request looks like when using an HTTP POST request:

```
POST http://fictitiousbank/transfer.cgi HTTP/1.1
Host: fictitiousbank
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; de; rv:1.9)
Gecko/2008052906 Firefox/3.6.2
```